Chapter 11: Getting Started with Pygame and Tractor Pong

Contents

Chapter 11: Getting Started with Pygame and Tractor Pong 1
How Games Work 2
  Game Loop 2
What is Pygame? 2
Install and Update Pygame with pip 2
Week 11 Tutorial 1: Hello Pygame 3
Pong 5
Week 11 Tutorial 2: Tractor Pong 1 - Draw Stationary Ball 5
Pygame Core Concepts 8
  Game Loops and Game States 9
  Quitting the Game Loop 10
  Event Objects 11
  Pixel Coordinates 12
  Colors 14
  FPS (Frames per Second) 16
  Screen double-buffering 17
Week 12 Tutorial 3: Tractor Pong 2 - Movement 17
  Delta Time 19
  Delta-time and frame rate 21
  Rect Object 21
  Animation 23
Week 13 Tutorial 4: Tractor Pong 3 - Bouncing and OOP 24
Week 14 Tutorial 5: Tractor Pong 4 - Add the Tractor 27
Week 15 Tutorial 6: Tractor Pong 5 - Collision Detection 32
Week 16 Tutorial 7: Tractor Pong 6 - Scoring and Music 33
  Assignment Submission 37

Time required: 90 minutes

How Games Work

Games are like films. A film is a collection of images played quickly in sequence (usually 24 frames per second). A 90-minute file would be about 129,600 images. Your brain stitches together the images into continuous movement. The only difference between movies and games is how each image is created.

Games work in a similar way. We create a lot of frames per second. We try to have at least 30 frames per second (more is better). The difference is that each image is dynamically generated as you play.

Game Loop

This is a typical game loop. This happens every time the game draws a new frame continuously and very fast, typically at least 30 frames per second.

1. Get player input and check for change.
2. We use input to move the player and update all elements.
3. All individual images are combined (player image, enemy images, ui graphics).
4. The final frame is displayed.

What is Pygame?

The Pygame library is the most well-known python library for making games. It’s not the most advanced or high-level library. It is not a game engine. Much of the work is done directly by the programmer.

It is simple and easy to learn (comparatively). Pygame is a great entry point into the world of graphics and game development.

Pygame is installed with Python. Pygame is a framework that includes several modules with functions for drawing graphics, playing sounds, handling mouse input, and other game like things. Pygame provides functions for creating programs with a **graphical user interface,** or **GUI** (pronounced, “gooey”).

Install and Update Pygame with pip

pip is a package manager for **Python**. It's a tool that allows you to install and manage additional libraries and dependencies that are not distributed as part of the standard library.

Install Pygame:

```
# Install pygame library
pip install pygame
```

If you are using Pygame on a regular basis, you will want to Update Pygame.

```
# Update out of date library
pip install pygame -U
```

Check for out of date libraries.

```
# Check for out of date libraries
pip list -o
```

Week 11 Tutorial 1: Hello Pygame

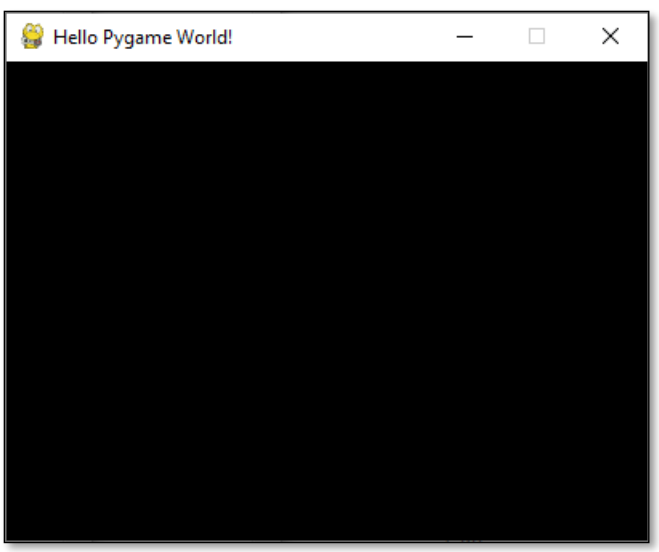
Video walkthrough: https://somup.com/c0eu3LyS71

Yet another Hello, World program. This time we do it in Pygame.

Type in the following code and save the file as **hello_pygame.py**.

```python
"""
    Name: hello_pygame.py
    Author:
    Date:
    Purpose: Create a blank Pygame window
"""
# Import pygame libraries
import pygame
from pygame.locals import *
# Import sys.exit to cleanly exit program
import sys

#--------------- INITIALIZE PYGAME ---------------#
# Initialize the pygame module
pygame.init()
# Create display window
SCREEN = pygame.display.set_mode((400, 300))
# Set caption for window
pygame.display.set_caption("Hello Pygame World!")

#--------------- INFINITE GAME LOOP --------------#
while True:
    # Listen for program events
    for event in pygame.event.get():

        # Closing the program window
        # causes the QUIT event to be fired
        if event.type == QUIT:
            pygame.quit()
            sys.exit()

    # Redraw the screen
    pygame.display.update()
```

Example run:



Yay! You've just made the world's most boring video game! It's just a blank window with Hello World! at the top of the window in the title bar.

Creating a window is the first step to making graphical games.

Pong

Imagine the year is 1973. All the talk is about a new arcade game which has been released by Atari. That game is called Pong.

It might not seem much of a game by today's standards. It was a massive hit in its day.

Don't be deceived, although a simple game, Pong covers a wide range of aspects of computer game programming. There is movement, control, collision detection, scoring, artificial intelligence. It's all in there!

Being able to program Pong is a doorway to being able to program a lot of other games.

Watch out! Once you start playing Pong you might find less time to program, as it is quite addictive!

We aren't going to do a complete Pong tutorial. We will get started with drawing and bouncing a ball around.

Week 11 Tutorial 2: Tractor Pong 1 - Draw Stationary Ball

Video walkthrough: https://somup.com/c0eu3LyS71

You can download the **tractor_pong_assets.zip** file attached to this assignment to an assets folder in this assignment folder.

CREATIVE TIP: Find an image of your own for the ball. Find your own assets as we go through this tutorial. Try to make this tutorial more your own and be creative with it. You can only learn so much by typing in a tutorial literally, experiment with it.

There are many websites with free open-source images. If you wish to change out the images:

1. Go to https://www.iconarchive.com
2. Search for an image you would like for your ball.
3. When you pick an image, Click **All Download Formats**.
4. Choose **PNG: 48px**

5. Download that file. Place in your assets folder.

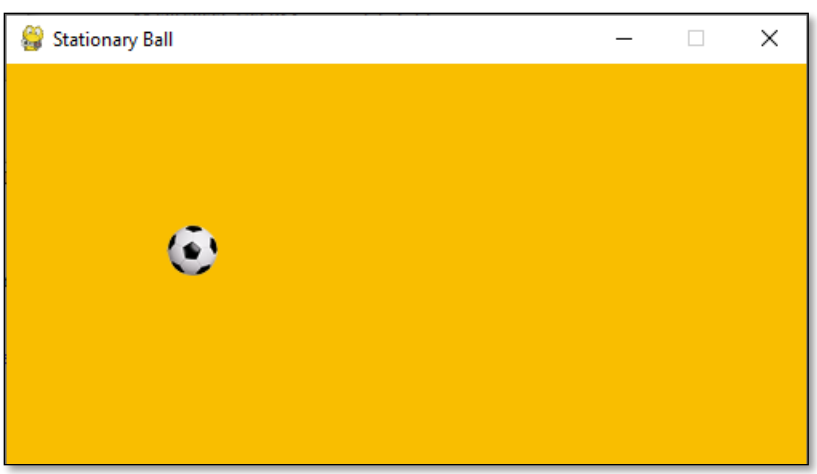
6. Either rename the file, or change the reference in the program.

Create a Python file named **tractor_pong_1.py** Add the following code.

```python
"""
    Name: tractor_pong_1.py
    Author:
    Date:
    Purpose: Draw a stationary ball
"""

# Import pygame library
import pygame

COUGAR_GOLD = (249, 190, 0)
# ------------------- INITIALIZE PYGAME -----------------------------------#
# Initialize pygame module for action
pygame.init()
# Set caption for window
pygame.display.set_caption("Tractor Pong")

# Create game surface: width x and height y are set as a tuple
surface = pygame.display.set_mode((500, 700))

# Load the ball image from the file system into a variable
ball = pygame.image.load("./assets/soccer_ball.png")

# Create a rectangle the same size as the ball
# rect is used to set the location of the ball
ball_rect = ball.get_rect()

# Initial postion of the ball rectangle x/left = 10, y/top = 10
ball_rect.left = 10
ball_rect.top = 10
```

```python
# ---------------------------- GAME LOOP ------------------------------------#
while True:
    """Infinite game loop"""
    # ----------------------- EVENTS ---------------------------------------#
    # Listen for program events
    for event in pygame.event.get():
        # Capture the QUIT event of closing the window
        if event.type == pygame.QUIT:
            # Quit Pygame
            pygame.quit()
            # Exit Python
            exit()

    # -------------------- DRAW ------------------------------------------#
    # Fill the display surface, Clears the previous screen
    # Comment out this line to see why is is necessary
    surface.fill(COUGAR_GOLD)

    # Draw the ball on the surface
    surface.blit(
        ball,       # Image to draw
        ball_rect   # Location to draw the image
    )

    # -------- COPY BACKBUFFER INTO VIDEO MEMORY ---------------------------#
    # Redraw the surface object
    pygame.display.update()
```

Example run:



The program uses **surface.fill(COUGAR_GOLD)** to clear the screen each time through the game loop. Comment out that line to see why.

Pygame Core Concepts

Let's start by explaining several core concepts related to the Pygame library and about creating games in general. Many of these concepts are transferable skills. If you switch to a more advanced game engine, many of these concepts will still hold true.

```
import pygame
```

The above code imports **pygame** and its modules into the program.

```
pygame.init()
```

This line is necessary whenever you want to use the Pygame library. Think of it as starting the pygame engine. It must be added before any other **pygame** function, or else an initialization error may occur.

```
# Create a display surface referenced by the surface reference variable
SURFACE = pygame.display.set_mode((700, 500))
```

This is a call to the **pygame.display.set_mode()** function, which returns the **pygame.Surface** object for the window. (Surface objects are described later.) Notice that we pass a tuple value of two integers to the function: **(700, 500)**. This tuple tells the **set_mode()** function how wide (x) and how high (y) to make the window in pixels. **(700, 500)** will make a window with a width of 700 pixels and height of 500 pixels.

An important aspect in games is individually accessing coordinates. To show a set of co-ordinates, you place both the X and Y values in a tuple, where the first integer is X and second integer is Y.

Both values start from the top-left hand side. X values increase from left to right, and Y values increase from top to bottom.

The coordinate values must be integers. A pixel is meant to represent the smallest possible area on a screen. There is no such thing as "half a pixel".

The **pygame.Surface** object (we will just call them Surface objects for short) returned is stored in a reference variable named **SURFACE**

```
# Sets the title bar caption
pygame.display.set_caption("Bouncing Ball")
```

This sets the caption text that will appear at the top of the window by calling the **pygame.display.set_caption()** function. The string value 'Bouncing Ball' is passed in this function call to make that text appear as the caption of the window.

Game Loops and Game States

Every game that has what is called a "Game Loop". The game loop runs forever until the game is closed. All games have some form of a game loop.

The Game Loop is where all the game events occur, update, and get drawn to the screen. Once the initial setup and initialization of variables is out of the way, the Game Loop begins where the program keeps looping over and over until an event of type **QUIT** occurs.

Changes in the game are not implemented until the **pygame.display.update()** has been called. Since games are constantly changing values, the update function is in the game loop, constantly updating.

```
#Game loop runs forever
while True:
    # Code
    # Code

    pygame.display.update()
```

This is a while loop that has a condition of simply the value **True**. This means that it never exits due to its condition evaluating to False. The only way the program execution will ever exit the loop is if a break statement is executed (which moves execution to the first line after the loop) or **sys.exit()** (which terminates the program). If a loop like this was inside a function, a return statement will also move execution out of the loop (as well as the function too).

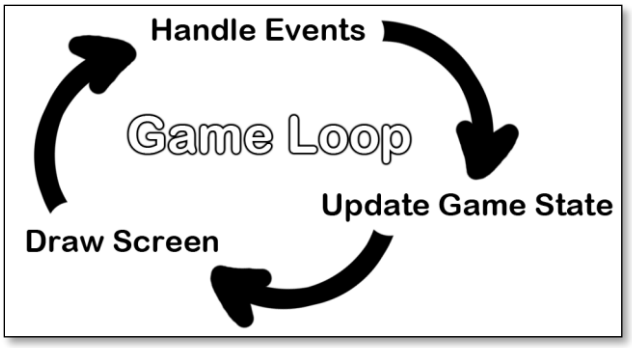
A game loop (also called a main loop) is a loop where the code does three things:

1. Handles events
2. Updates the game state
3. Draws the game state to the screen

The game state is simply a way of referring to a set of values for all the variables in a game program. In many games, the game state includes the values in the variables that tracks the player's health and position, the health and position of any enemies, which marks have been made on a board, the score, or whose turn it is. Whenever something happens like the player taking damage (which lowers their health value), or an enemy moves somewhere, or something happens in the game world we say that the game state has changed.

If you've ever played a game that let you save, the "save state" is the game state at the point that you've saved it. In most games, pausing the game will prevent the game state from changing.

Since the game state is usually updated in response to events (such as mouse clicks or keyboard presses) or the passage of time, the game loop is constantly checking and re-checking many times a second for any new events that have happened. Inside the main loop is code that looks at which events have been created (with Pygame, this is done by calling the **pygame.event.get()** function). The main loop also has code that updates the game state based on which events have been created. This is usually called event handling.



Quitting the Game Loop

Every game loop must have an end point, or some action that triggers the end point (such as clicking the quit button), or else your game will run indefinitely.

```
while True:
    pygame.display.update()
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
```

We call both **pygame.quit()** and **sys.exit()** to close the pygame window and the python script respectively.

Event objects have a member variable (also called attributes or properties) named **type** which tells us what kind of event the object represents. Pygame has a constant variable for each of possible types in the **pygame.locals** modules. Line 9 checks if the Event object's type is equal to the constant QUIT.

If the Event object is a quit event, then the **pygame.quit()** and **sys.exit()** functions are called. The **pygame.quit()** function is sort of the opposite of the **pygame.init()** function:

it runs code that deactivates the Pygame library. Your programs should always call **pygame.quit()** before they call **sys.exit()** to terminate the program..

We have no if statements that run code for other types of Event objects. There isn't any event-handling code for when the user clicks the mouse, presses keyboard keys, or causes any other type of Event objects to be created. The user can do things to create these Event objects, but it doesn't change anything in the program because the program does not have any event-handling code for these types of Event objects. After the for loop on is done handling all the Event objects that have been returned by **pygame.event.get()**, the program execution continues to these final lines. We will add more event handling as we grow our program.

```
# Redraw the surface object
pygame.display.update()
```

This line calls the **pygame.display.update()** function, which draws the Surface object returned by **pygame.display.set_mode()** to the screen (remember we stored this object in the **window** variable). Since the Surface object hasn't changed (for example, by some of the drawing functions that are explained later in this chapter), the same image is redrawn to the screen each time **pygame.display.update()** is called.

That is the entire program. After this line is done, the infinite while loop starts again from the beginning. This program does nothing besides make a window appear on the screen, constantly check for a **QUIT** event, and then redraws the unchanged window to the screen over and over and over again.

Event Objects

Any time the user does one of several actions such as pressing a keyboard key or moving the mouse on the program's window, a `pygame.event.Event` object is created by the Pygame library to record this "event". (This is a type of object called Event that exists in the event module, which itself is in the pygame module.) We can find out which events have happened by calling the **pygame.event.get()** function, which returns a list of **pygame.event.Event** objects (which we will just call Event objects for short).

The list of Event objects will be for each event that has happened since the last time the **pygame.event.get()** function was called.

```
# Watch for any program events
for event in pygame.event.get():

    # Closing the program by clicking the X
    # causes the QUIT event to be fired
    if event.type == pygame.QUIT:

        # Quit Pygame
        pygame.quit()

        # Stop Python script
        sys.exit()
```

This is a for loop that will iterate over the list of Event objects that was returned by **pygame.event.get()**. On each iteration through the for loop, a variable named event will be assigned the value of the next event object in this list. The list of Event objects returned from **pygame.event.get()** will be in the order that the events happened. If the user clicked the mouse and then pressed a keyboard key, the Event object for the mouse click would be the first item in the list and the Event object for the keyboard press would be second. If no events have happened, then **pygame.event.get()** will return a blank list.

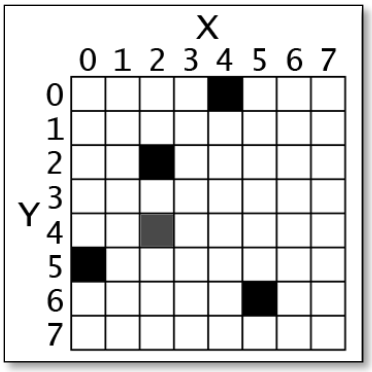
An "Event" occurs when the user performs a specific action, such as clicking their mouse or pressing a keyboard button. Pygame records every event that occurs.

One of the many attributes (or properties) held by event objects is type. The type attribute tells us what kind of event the object represents.

If you take a look at the example shown earlier, you'll see we used **event.type == QUIT** to determine whether the game is to be closed or not.

Pixel Coordinates

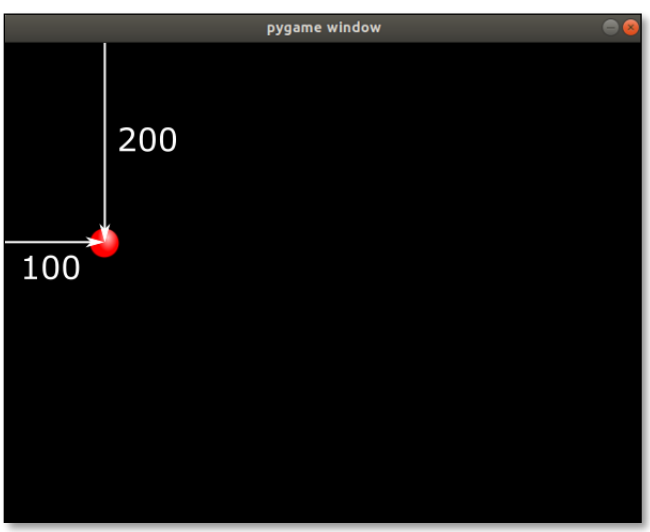
The window that the "Bouncing Ball" program creates is composed of little square dots on the screen called pixels. Each pixel starts off as black but can be set to a different color. Imagine that instead of a Surface object that is 300 pixels wide and 300 pixels tall, we just had a Surface object that was 8 pixels by 8 pixels. If that tiny 8x8 Surface was enlarged so that each pixel looks like a square in a grid, and we added numbers for the X and Y axis, then a good representation of it could look something like this:



We can refer to a specific pixel by using a Cartesian Coordinate system. Each column of the X-axis and each row of the Y-axis will have an "address" that is an integer from 0 to 7 so that we can locate any pixel by specifying the X and Y axis integers.

For example, in the above 8x8 image, we can see that the pixels at the XY coordinates (4, 0), (2, 2), (0, 5), and (5, 6) have been painted black, the pixel at (2, 4) has been painted gray, while all the other pixels are painted white. XY coordinates are also called points. If you've taken a math class and learned about Cartesian Coordinates, you might notice that the Y-axis starts at 0 at the top and then increases going down, rather than increasing as it goes up. This is just how Cartesian Coordinates work in Pygame (and almost every programming language).

The Pygame framework often represents Cartesian Coordinates as a tuple of two integers, such as (4, 0) or (2, 2). The first integer is the X coordinate and the second is the Y coordinate.



Colors

Colors are a big part of any game development framework or engine.

Pygame uses the RGB system of colors. This stand for Red, Green and Blue respectively. These three colors combined (in varying ratios) are used to create all the colors you see on computers, or any device that has a screen.

Some of these colors are predefined in Pygame. See the **pygame_color_picker.py** program attached to this assignment.

There are three primary colors of light: red, green and blue. (Red, blue, and yellow are the primary colors for paints and pigments, but the computer monitor uses light, not paint.) By combining different amounts of these three colors you can form any other color.

In Pygame, we represent colors with tuples of three integers. The first value in the tuple is how much red is in the color. An integer value of 0 means there is no red in this color, and a value of 255 means there is the maximum amount of red in the color. The second value is for green and the third value is for blue. These tuples of three integers used to represent a color are often called RGB values.

Because you can use any combination of 0 to 255 for each of the three primary colors, this means Pygame can draw 16,777,216 different colors (that is, 256 x 256 x 256 colors).

For example, we will create the tuple (0, 0, 0) and store it in a variable named BLACK. With no amount of red, green, or blue, the resulting color is completely black. The color black is the absence of any color. The tuple (255, 255, 255) for a maximum amount of red, green,

and blue to result in white. The color white is the full combination of red, green, and blue. The tuple (255, 0, 0) represents the maximum amount of red but no amount of green and blue, so the resulting color is red. Similarly, (0, 255, 0) is green and (0, 0, 255) is blue.

The values for each color range from 0 – 255, a total of 256 values. You can find the total number of possible color combinations by evaluating 256 x 256 x 256, which results in a value well over 16 million.

You can mix the amount of red, green, and blue to form other colors. Here are the RGB values for a few common colors:

Color	RGB Values
Aqua	(0, 255, 255)
Black	(0, 0, 0)
Blue	(0, 0, 255)
Fuchsia	(255, 0, 255)
Gray	(128, 128, 128)
Green	(0, 128, 0)
Lime	(0, 255, 0)
Maroon	(128, 0, 0)
Navy Blue	(0, 0, 128)
Olive	(128, 128, 0)
Purple	(128, 0, 128)
Red	(255, 0, 0)
Silver	(192, 192, 192)
Teal	(0, 128, 128)
White	(255, 255, 255)
Yellow	(255, 255, 0)

To use colors in Pygame, we first create Color objects using RGB values. RGB values must be in a tuple format, with three values, each corresponding to a respective color. Here are some examples below showing some color objects in pygame.

```
BLACK = pygame.Color(0, 0, 0)          # Black
WHITE = pygame.Color(255, 255, 255)    # White
GREY = pygame.Color(128, 128, 128)     # Grey
RED = pygame.Color(255, 0, 0)          # Red
HUSKER_RED = (227, 25, 55)
HUSKER_CREAM = (253, 242, 217)
COUGAR_BLUE = (0, 58, 112)
COUGAR_GOLD = (249, 190, 0)
```

The **fill(color)** method is used to fill in objects. For instance, assigning a rectangle the color green will only turn the borders green. If you use the **fill()** method and pass a green color object, the rectangle will become completely green.

FPS (Frames per Second)

Computers are extremely fast and can complete millions of loop cycles in under a second. Obviously, this is a bit fast for us humans. As reference, movies are run at 24 frames per second. Anything less than that will have obvious stutter to it, and values over 100 may cause the things to move too fast for us to see.

The frame rate or refresh rate is the number of pictures that the program draws per second. It is measured in FPS or frames per second. (On computer monitors, the common name for FPS is hertz. Many monitors have a frame rate of 60 hertz, or 60 frames per second.) A low frame rate in video games can make the game look choppy or jumpy. If the program has too much code to run to draw to the screen frequently enough, then the FPS goes down.

If we do not create a limitation, the computer will execute the game loop as many times as in can within a second. To limit it we use the **tick(fps)** method where **fps** is an integer. The **tick()** method belongs to the **pygame.time.Clock** class and must be used with an object of this class.

```
CLOCK = pygame.time.Clock()
CLOCK.tick(60)
```

The frames per second can vary from game to game, depending on how it was designed. A good value to aim for is between 30 – 60. Keep in mind, that if you create a rather complex and heavy game the computer might not be able to run it well at higher frames.

A **pygame.time.Clock** object can help us make sure our program runs at a certain maximum FPS. This **Clock** object will ensure that our game programs don't run too fast by putting in small pauses on each iteration of the game loop. If we didn't have these pauses, our game program would run as fast as the computer could run it. This is often too fast for the player, and as computers get faster they would run the game faster too. A call to the `tick()` method of a `Clock` object in the game loop can make sure the game runs at the same speed no matter how fast of a computer it runs on.

```
CLOCK = pygame.time.Clock()
```

The Clock object's `tick()` method should be called at the very end of the game loop, after the call to `pygame.display.update()`. The length of the pause is calculated based on how long it has been since the previous call to `tick()`, which would have taken place at the end of the previous iteration of the game loop. (The first time the `tick()` method is called, it doesn't pause at all.)

All you need to know is that you should call the `tick()` method once per iteration through the game loop at the end of the loop. Usually this is right after the call to `pygame.display.update()`.

```
CLOCK.tick(FPS)
```

Try modifying the `FPS` constant variable to run the same program at different frame rates. Setting it to a lower value would make the program run slower. Setting it to a higher value would make the program run faster.

Screen double-buffering

The image you see on your screen is stored in memory as a pixel buffer. This memory area, called the video memory, stores the color of every pixel on the screen.

We don't draw directly to the video memory. That is because the drawing process involves clearing the memory, then drawing the individual sprites, one by one. This would cause flickering if we did it in the video memory.

Instead, we use a second memory buffer that is just ordinary memory (it doesn't affect the display). This is sometimes called the back buffer. We draw our sprites into that buffer. When we have finished drawing and every pixel is set to its final color, we call flip, which copies that memory buffer into the actual video memory. This is called double-buffering.

You don't need to worry about this, Pygame sorts most of it out. You just need to remember:

1. Draw all your sprites using the screen object (this represents the back buffer).
2. When all the drawing is complete, call **pygame.display.update()** to copy the complete buffer into video memory.

Week 12 Tutorial 3: Tractor Pong 2 - Movement

Video walkthrough: https://somup.com/c0fnD74hzs

Time to move our ball. Open **tractor_pong_1.py** Save as **tractor_pong_2.py** Add the following code. There are only a few changes on the lower part of the program.

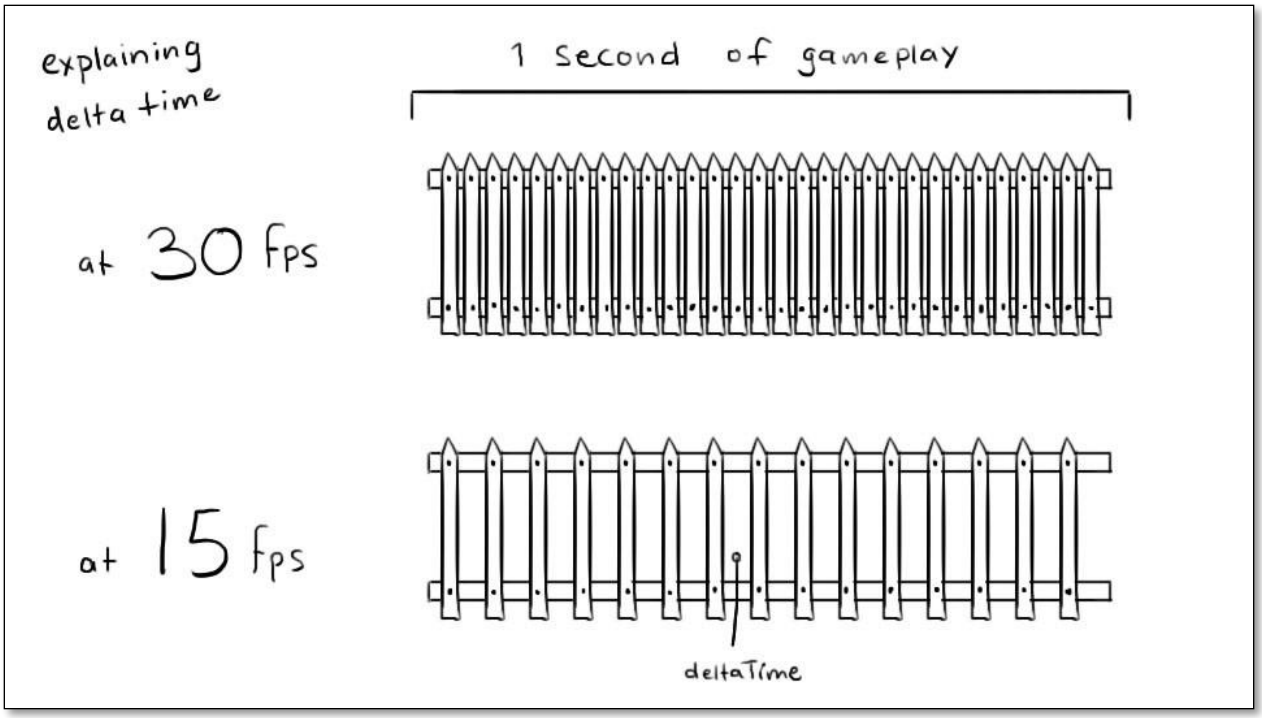
```python
"""
    Name: tractor_pong_2.py
    Author:
    Date:
    Purpose: The ball moves across the screen
"""

# Import pygame library
import pygame
# Import sys.exit to cleanly exit program
from sys import exit

COUGAR_GOLD = (249, 190, 0)
# Set screensize constants
WIDTH = 500
HEIGHT = 700
# ------------------- INITIALIZE PYGAME -----------------------------------#
# Initialize pygame for action
pygame.init()
# Set caption for window
pygame.display.set_caption("Tractor Pong")

# Create game surface: width x and height y are set as a tuple
surface = pygame.display.set_mode((500, 700))

# The CLOCK object manages how fast the game runs
clock = pygame.time.Clock()

# Load the ball image from the file system into a variable
ball = pygame.image.load("./assets/soccer_ball.png")

# Create a rectangle the same size as the ball
# rect is used to set the location of the ball
ball_rect = ball.get_rect()

# Initial postion of the ball rectangle x/left = 10, y/top = 10
ball_rect.left = 10
ball_rect.top = 10

# Ball speed in pixels as an x, y list
speed = [0.2, 0.2]
```

```python
# ---------------------------- GAME LOOP ----------------------------------#
while True:
    """Infinite game loop"""
    # Change in delta time for each frame
    # Multiplying movement by dt allows all computers
    # to run the game at the same speed
    # 100 frames per second
    dt = clock.tick(100)

    # ----------------------- EVENTS ---------------------------------------#
    # Listen for program events
    for event in pygame.event.get():
        # Capture the QUIT event of closing the window
        if event.type == pygame.QUIT:
            # Quit Pygame
            pygame.quit()
            # Exit Python
            exit()

    # ----------------------- UPDATE BALL ----------------------------------#
    # Move ball rect using speed list x = speed[0], y = speed[1]
    ball_rect.left += speed[0] * dt
    ball_rect.top += speed[1] * dt

    # -------------------- DRAW -------------------------------------------#
    # Fill the display surface, Clears the previous screen
    # Comment out this line to see why is is necessary
    surface.fill(COUGAR_GOLD)

    # Draw the ball on the surface
    surface.blit(
        ball,       # Image to draw
        ball_rect   # Location to draw the image
    )

    # -------- COPY BACKBUFFER INTO VIDEO MEMORY ---------------------------#
    # Redraw the surface object
    pygame.display.update()
```

When this program is run, the window is displayed until the user closes the window. The ball keeps moving until it disappears off the screen.

Delta Time

We added a Clock object to keep our game running on time. The ball is moving at 100 frames per second. We are using a game programming concept called delta time to keep the game play the same on any device.



Delta time is a term used for variably updating scenery based on the elapsed time since the game last updated, (the previous "frame") which will vary depending on the speed of the computer, and how much work needs to be done in the program at any given time.

This is done by calling a timer every frame per second that holds the time between now and last call in milliseconds. The resulting number (delta time) is used to calculate how far, for instance, a video game character would have travelled during that time. This results in the character taking the same amount of real-world time to move across the screen regardless of the rate of update, and whether the delay is caused by lack of processing power or a slow internet connection.

This avoids gameplay slowing down or speeding up depending on the complexity of what is happening at any given time, which would make for an inconsistent, jarring experience (e.g. time slowing down the more characters walk onto the screen, or running too fast because only one character is on screen).

Big enough delays will eventually negatively affect the gameplay experience, but using Delta Time keeps the gameplay consistent so long as the computer and internet connection meet the minimum hardware requirements of the game.

Delta-time and frame rate

Delta-time and frame rate are not always related. Video games fall into one of two categories regarding frame rate: frame rate dependent or frame rate independent. Frame rate Dependent games have a frame rate that varies based on the computer running the software. For example, if a frame rate dependent game runs at 300 frames per second (fps) on a computer with a refresh rate of 120 hertz (Hz), then it would run at 150 fps on a computer with a refresh rate of 60 Hz. A standard delta-time expression can create pause screens and intentional slow-motion effects in frame rate-dependent games. Standard delta-time formulas are seldom used for standard game-play because the delta-time between frames varies greatly depending on the refresh rate of the computer on which it is running.

If a game is frame rate independent, the frame rate is preset and runs identically across all computers regardless of their specifications. Frame rate independence limits the maximum graphics quality to make the game available to more consumers. Frame rate independence is particularly popular in mobile games and games optimized for lower-end computers such as Chromebooks. In frame rate independent games, the delta-time between frames is consistent across the entire game. This standardization means that one delta-time expression can create a consistent frame rate for all users on all types of computers.

https://en.wikipedia.org/wiki/Delta_timing

Rect Object

Instead of drawing the ball directly to the screen, we create a Rect object that we draw the ball in.

The code below creates a Rect object that is the same size as the ball..

```
ball = pygame.image.load("ball_sports.png")
ball_rect = ball.get_rect()
```

The handy thing about this is that the Rect object automatically calculates the coordinates for other features of the rectangle. Rect makes moving other objects easier to find and move around the game screen.

If you need to know the X coordinate of the right edge of the pygame.Rect object we stored in the ball_rect variable, you access the Rect object's right attribute:

```
ball_rect.right
210
```

The Pygame code for the Rect object automatically calculated that if the left edge is at the X coordinate 10 and the rectangle is 200 pixels wide, then the right edge must be at the X coordinate 210. If you reassign the right attribute, all the other attributes are automatically recalculated. If we used drawing objects, we would have to calculate this manually.

```
ball_rect.right = 350
ball_rect.left
150
```

Here's a list of all the attributes that pygame.Rect objects provide:

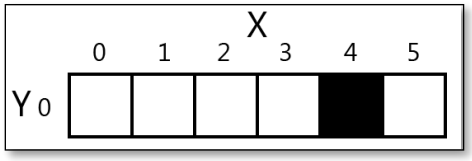
Attribute Name	**Description**
ball_rect.left	The int value of the X-coordinate of the left side of the rectangle.
ball_rect.right	The int value of the X-coordinate of the right side of the rectangle.
ball_rect.top	The int value of the Y-coordinate of the top side of the rectangle.
ball_rect.bottom	The int value of the Y-coordinate of the bottom side.
ball_rect.centerx	The int value of the X-coordinate of the center of the rectangle.
ball_rect.centery	The int value of the Y-coordinate of the center of the rectangle.
ball_rect.width	The int value of the width of the rectangle.
ball_rect.height	The int value of the height of the rectangle.
ball_rect.size	A tuple of two ints: (width, height)
ball_rect.topleft	A tuple of two ints: (left, top)
ball_rect.topright	A tuple of two ints: (right, top)
ball_rect.bottomleft	A tuple of two ints: (left, bottom)

ball_rect.bottomright	A tuple of two ints: (right, bottom)
ball_rect.midleft	A tuple of two ints: (left, centery)
ball_rect.midright	A tuple of two ints: (right, centery)
ball_rect.midtop	A tuple of two ints: (centerx, top)
ball_rect.midbottom	A tuple of two ints: (centerx, bottom)

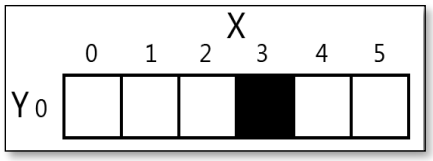
Animation

Now that we know how to get the Pygame framework to draw to the screen, let's learn how to make animated pictures. A game with only still, unmoving images is dull. (Sales of the game "Look At This Rock" have been disappointing.)

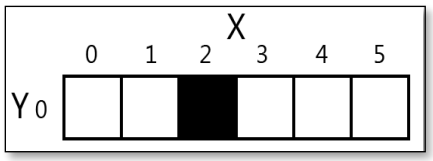
Animated images are the result of drawing an image on the screen, then a split second later drawing a slightly different image on the screen. Imagine the program's window was 6 pixels wide and 1 pixel tall, with all the pixels white except for a black pixel at 4, 0. It would look like this:



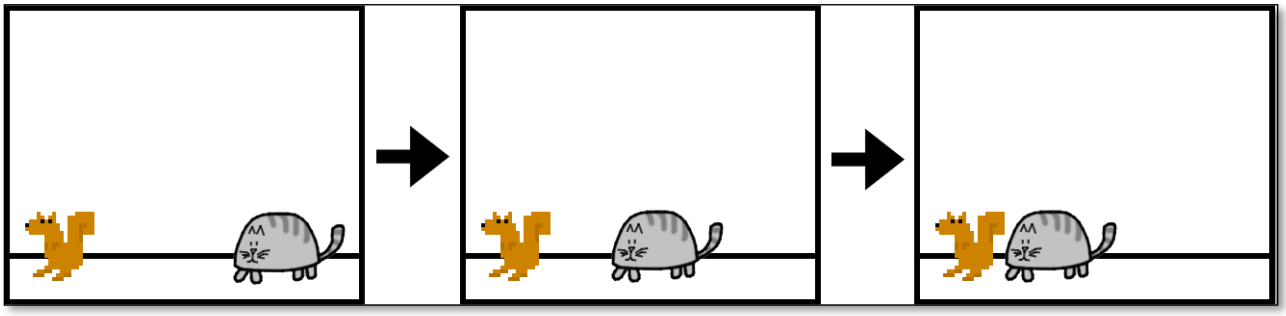
If you changed the image so that 3, 0 was black and 4, 0 was white, it would look like this:



To the user, it looks like the black pixel has "moved" over to the left. If you redrew the window to have the black pixel at 2, 0, it would continue to look like the black pixel is moving left:



It may look like the black pixel is moving, but this is just an illusion. To the computer, it is just showing three different images that each just happen to have one black pixel. Consider if the three following images were rapidly shown on the screen:



To the user, it would look like the cat is moving towards the squirrel. But to the computer, they're just a bunch of pixels. The trick to making believable looking animation is to have your program draw a picture to the window, wait a fraction of a second, and then draw a slightly different picture.

Week 13 Tutorial 4: Tractor Pong 3 - Bouncing and OOP

Video walkthrough: https://wncc.screencasthost.com/watch/c0fQnlVaks4

Let's bounce the ball around the screen. We are going to refactor the code to OOP (Object Oriented Programming). OOP allows for more flexible expansion in the future.

Open **tractor_pong_2.py** Save as **tractor_pong_3.py** Change the code to the following:

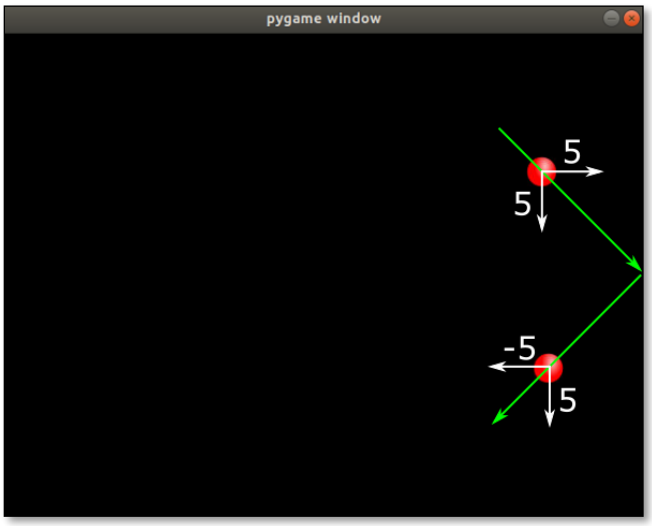
```python
"""
    Name: tractor_pong_3.py
    Author:
    Date:
    Purpose: Bounce a ball around the screen
"""

# Import pygame library
import pygame
# Import sys.exit to cleanly exit program
from sys import exit


class TractorPong:
    # ------------------- INITIALIZE PYGAME --------------------------------#
    def __init__(self):
        self.COUGAR_GOLD = (249, 190, 0)
        # Set screensize constants
        self.WIDTH = 500
        self.HEIGHT = 700

        # Initialize pygame for action
        pygame.init()
        # Set caption for window
        pygame.display.set_caption("Tractor Pong")

        # Create a game surface: width x and height y are set as a tuple
        self.surface = pygame.display.set_mode((self.WIDTH, self.HEIGHT))

        # CLOCK object manages how fast the game runs
        self.clock = pygame.time.Clock()

    # ------------------ LOAD ASSETS ---------------------------------------#
        # Load the ball image from the file system into a variable
        self.ball = pygame.image.load(
            "./assets/soccer_ball.png").convert_alpha()

        # Create a rectangle the same size as the ball
        # rect is used to set the location of the ball
        self.ball_rect = self.ball.get_rect()

        # Initial postion of the ball rectangle x/left = 10, y/top = 10
        self.ball_rect.left = 10
        self.ball_rect.top = 10

        # Ball speed in pixels as an x, y list
        self.speed = [0.2, 0.2]
```

```python
# ---------------------------- GAME LOOP ----------------------------------#
    def run_game(self):
        while True:
            """Infinite game loop"""
            # Change in delta time for each frame
            # Multiplying movement by dt allows all computers
            # to run the game at the same speed
            # 100 frames per second
            dt = self.clock.tick(100)

            # -------------------------- EVENTS ----------------------------#
            # Listen for program events
            for event in pygame.event.get():
                # Closing the program window
                # causes the QUIT event to be fired
                if event.type == pygame.QUIT:
                    # Quit Pygame
                    pygame.quit()
                    # Exit Python
                    exit()

            # -------------------------- UPDATE ----------------------------#
            # Check for collision with left or right wall
            if self.ball_rect.left < 0 or self.ball_rect.right > self.WIDTH:
                # Reverse [0] x direction
                self.speed[0] = -self.speed[0]

            # Check for collision with top or bottom wall
            if self.ball_rect.top < 0 or self.ball_rect.bottom > self.HEIGHT:
                # Reverse [1] y direction
                self.speed[1] = -self.speed[1]

            # Move ball rect using speed list x = speed[0], y = speed[1]
            self.ball_rect.left += self.speed[0] * dt
            self.ball_rect.top += self.speed[1] * dt

            # -------------------------- DRAW ------------------------------#
            # Fill the display surface, Clears the previous screen
            # Comment out this line to see why is is necessary
            self.surface.fill(self.COUGAR_GOLD)

            # Draw the ball on the backbuffer
            self.surface.blit(
                self.ball,       # Image to draw
                self.ball_rect   # Location to draw the image
            )

            # ------- COPY BACKBUFFER INTO VIDEO MEMORY --------------------#
            pygame.display.update()


# Initialize program object and start game
tractor_pong = TractorPong()
tractor_pong.run_game()
```

```
# Check for collision with left or right wall
if ballrect.left < 0 or ballrect.right > WIDTH:
    # Reverse x direction
    speed[0] = -speed[0]
```

If the ball's left edge moves past the left edge of the screen, the direction reverses by changing the speed to the negative of what it currently is. If the current speed is +2, when it is changed to a negative, it reverses direction. This works the same with each edge of the screen.

The image below shows how the x direction will reverse.



Week 14 Tutorial 5: Tractor Pong 4 - Add the Tractor

What better paddle to use in Pong than a tractor! The code is refactored into methods.

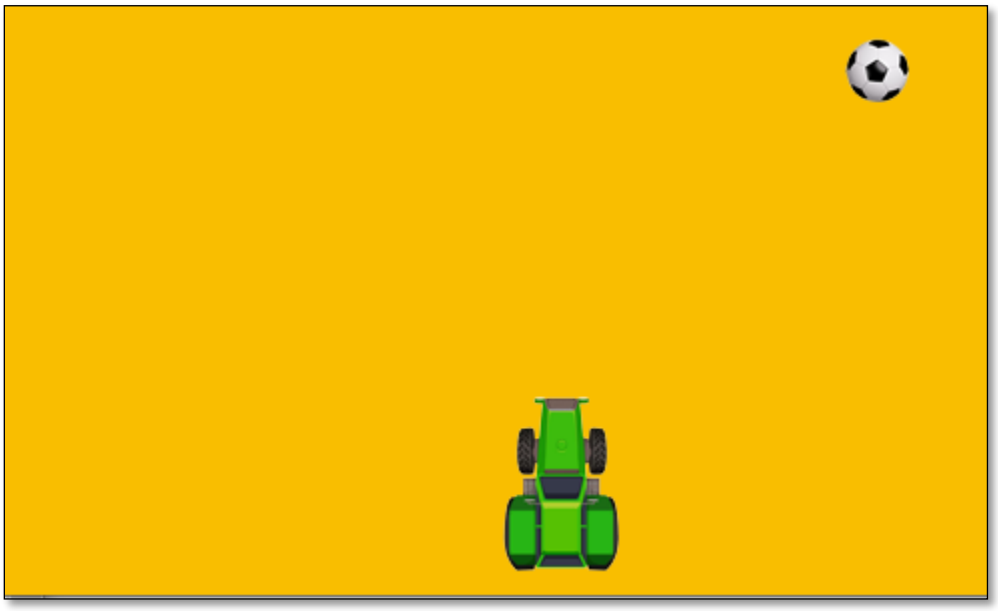
CREATIVE TIP: Come on, you can find a more creative image than a tractor to use as a ping pong paddle.

```
    # ------------------ LOAD ASSETS ---------------------------------------#
        # Load the images from the file system into a variable
        self.ball_img = pygame.image.load(
            "./assets/soccer_ball.png").convert_alpha()
        self.tractor_img = pygame.image.load(
            "./assets/green_tractor.png").convert_alpha()

        # Create a rectangle the same size as the image
        # rect is used to set the location of the image
        self.ball_rect = self.ball_img.get_rect()
        self.tractor_rect = self.tractor_img.get_rect()

        # Initial postion of the ball rectangle x/left = 10, y/top = 10
        self.ball_rect.left = 10
        self.ball_rect.top = 10

        # Initial location of the tractor
        self.tractor_rect.left = self.WIDTH // 2
        self.tractor_rect.top = self.HEIGHT - 100

        # Speed in pixels as an x, y list
        self.ball_speed = [0.2, 0.2]
        self.tractor_speed = 0.4
```

```
# ---------------------------- GAME LOOP ------------------------------------#
    def run_game(self):
        """Infinite game loop"""
        while True:
            # Change in delta time for each frame
            # Multiplying movement by dt allows all computers
            # to run the game at the same speed
            # 100 frames per second
            self.dt = self.clock.tick(100)
            self.events()
            self.update_tractor()
            self.update_ball()
            self.draw()

# -------------------------- EVENTS ----------------------------------------#
    def events(self):
        """Listen for and handle program events"""
        for self.event in pygame.event.get():
            # Closing the program window
            # causes the QUIT event to be fired
            if self.event.type == pygame.QUIT:
                # Quit Pygame
                pygame.quit()
                # Exit Python
                exit()

```

```
83  # ------------------------- UPDATE TRACTOR ---------------------------------#
84      def update_tractor(self):
85          # Capture key pressed events into a list
86          pressed_keys = pygame.key.get_pressed()
87          # Move the tractor in the direction of the cursor keys
88          if pressed_keys[pygame.K_LEFT]:
89              self.tractor_rect.left -= self.tractor_speed * self.dt
90          if pressed_keys[pygame.K_RIGHT]:
91              self.tractor_rect.left += self.tractor_speed * self.dt
92
93  # ------------------------- UPDATE BALL ------------------------------------#
94      def update_ball(self):
95          # Check for collision with left or right wall
96          if self.ball_rect.left < 0 or self.ball_rect.right > self.WIDTH:
97              # Reverse [0] x direction
98              self.ball_speed[0] = -self.ball_speed[0]
99
100         # Check for collision with top or bottom wall
101         if self.ball_rect.top < 0 or self.ball_rect.bottom > self.HEIGHT:
102             # Reverse [1] y direction
103             self.ball_speed[1] = -self.ball_speed[1]
104
105         # Move ball rect using speed list
106         # x = speed[0], y = speed[1]
107         self.ball_rect.left += self.ball_speed[0] * self.dt
108         self.ball_rect.top += self.ball_speed[1] * self.dt
109
110 # --------------------------- DRAW -----------------------------------------#
111     def draw(self):
112         # Fill the display surface to clear the previous screen
113         # Comment out this line to see why is is necessary
114         self.surface.fill(self.COUGAR_GOLD)
115
116         # Draw the ball on the backbuffer
117         self.surface.blit(
118             self.ball_img,      # Image to draw
119             self.ball_rect      # Location to draw the image
120         )
121
122         # Draw the tractor on the backbuffer
123         self.surface.blit(
124             self.tractor_img,   # Image to draw
125             self.tractor_rect   # Location to draw the image
126         )
127         # ------- COPY BACKBUFFER INTO VIDEO MEMORY ------------------------#
128         # Copy the backbuffer into video memory
129         pygame.display.update()
130
131
132 # Initialize program object and start game
133 tractor_pong = TractorPong()
134 tractor_pong.run_game()
```

Example run:



You can use your left and right cursor keys to move the tractor back and forth.

Week 15 Tutorial 6: Tractor Pong 5 - Collision Detection

Let's fence the tractor in and get some collision detection going. We will make only a few changes to the code.

The **tractor_rect** properties keep track of where the tractor is. If it moves outside the surface boundary, the keys won't do anything.

```
# ---------------------------- GAME LOOP ----------------------------------#
    def run_game(self):
        """Infinite game loop"""
        while True:
            # Change in delta time for each frame
            # Multiplying movement by dt allows all computers
            # to run the game at the same speed
            # 100 frames per second
            self.dt = self.clock.tick(100)
            self.events()
            self.update_tractor()
            self.update_ball()
            self.check_collision()
            self.draw()
```

```
# ------------------------- UPDATE TRACTOR --------------------------------#
    def update_tractor(self):
        # Capture key pressed events into a list
        pressed_keys = pygame.key.get_pressed()
        # Is the tractor on the screen?
        # These if statements prevent the tractor
        # from going off of the screen
        # Decide which way to go, left or right
        if self.tractor_rect.left > 0:
            if pressed_keys[pygame.K_LEFT]:
                self.tractor_rect.left -= self.tractor_speed * self.dt
        if self.tractor_rect.right < self.WIDTH:
            if pressed_keys[pygame.K_RIGHT]:
                self.tractor_rect.left += self.tractor_speed * self.dt
```

```
# ---------------------- CHECK COLLISIOM ----------------------------------#
    def check_collision(self):
        # Check for collision between two rects
        # Does the ball collide with the tractor,
        # If so, reverse the ball y direction [1]
        if self.tractor_rect.colliderect(self.ball_rect):
            # Reverse y direction
            self.ball_speed[1] = -self.ball_speed[1]
```

Week 16 Tutorial 7: Tractor Pong 6 - Scoring and Music

Add the following code to the end of the init method.

With some sound, scoring and speed up of the ball, we have a real game.

CREATIVE TIP: Find your own music! There are lots of free wav files on the web.

```
        # Speed in pixels as an x, y list
        self.ball_speed = [0.2, 0.2]
        self.tractor_speed = 0.4

        # Keep track of score
        self.score = 0

        self.ball_hit = pygame.mixer.Sound("./assets/ball.wav")
        self.game_over = pygame.mixer.Sound(
            "./assets/tractor_driving_game_over.wav")
        # Set volume for sound effect in range 0.0 - 1.0
        pygame.mixer.Sound.set_volume(self.game_over, .5)

        # Load and play background music
        pygame.mixer.music.load("./assets/tractor_driving.wav")
        # Set volume for music in range 0.0 - 1.0
        pygame.mixer.music.set_volume(0.7)

        # Create fonts for game over screen
        self.font_small = pygame.font.SysFont("Verdana", 20)
        self.font_large = pygame.font.SysFont("Verdana", 60)

        # Only allow these events to be captured
        # This helps optimize the game for slower computers
        pygame.event.set_allowed(
            [
                pygame.QUIT,
                pygame.KEYDOWN,
                pygame.KEYUP,
                pygame.MOUSEBUTTONUP
            ]
        )
```

```python
# ---------------------------- GAME LOOP -----------------------------------#
    def run_game(self):
        """Infinite game loop"""
        # Stop any other music from playing
        pygame.mixer.stop()
        # Play background game music in continious loop from the beginning
        pygame.mixer.music.play(-1, 0.0)
        while True:
            # Change in delta time for each frame
            # Multiplying movement by dt allows all computers
            # to run the game at the same speed
            # 100 frames per second
            self.dt = self.clock.tick(100)
            self.events()
            self.update_tractor()
            self.update_ball()
            self.check_collision()
            self.draw()
```

```python
# ---------------------- CHECK COLLISION -----------------------------------#
    def check_collision(self):
        """Check for collision between two rects"""
        # The ball has to be above the tractor to collide
        # Does the ball collide with the tractor?
        # If so, reverse the ball y direction [1]
        if self.tractor_rect.colliderect(
            self.ball_rect
        ) and self.ball_rect.bottom < self.tractor_rect.top + 4:

            # Reverse y direction
            self.ball_speed[1] = -self.ball_speed[1]

            # Increase speed by 10% each time the ball is hit
            self.ball_speed[0] = self.ball_speed[0] * 1.1
            self.ball_speed[1] = self.ball_speed[1] * 1.1

            # Increase score by 1
            self.score = self.score + 1
            pygame.mixer.Sound.play(self.ball_hit)
```

Drawing text and buttons on the surface in Pygame takes a bit of code. All of this is required to pause the game and display a reset screen.

```python
# ----------------------------- RESET GAME ---------------------------------#
    def reset_game(self):
        """Reset the game back to the defaults to start over."""
        # Stop background music
        pygame.mixer.music.stop()
        # Play game_over music until the user clicks a button
        pygame.mixer.Sound.play(self.game_over, loops=-1)

        self.surface.fill(self.COUGAR_GOLD)

        # ------------------- DRAW SCORE ---------------------------------- #
        # Render score before drawing it on the surface
        score_display = self.font_small.render(
            f"{self.score}",     # Score
            True,                # Antialiasing true
            "black"              # Font color
        )
        # Draw score on the surface
        self.surface.blit(score_display, (10, 10))

        # ------------------- DRAW GAME OVER ------------------------------ #
        game_over_display = self.font_large.render(
            "Game Over!",     # Score
            True,             # Antialiasing true
            "black"           # Font color
        )
        # Get rect to center text on screen
        text_rect = game_over_display.get_rect(
            center=(self.WIDTH // 2, self.HEIGHT // 2))

        # Draw Game Over text on the screen
        self.surface.blit(
            game_over_display,
            text_rect
        )

        # ------------------- DRAW REPLAY BUTTON -------------------------- #
        replay_btn = pygame.draw.rect(
            self.surface,
            "black",
            [self.WIDTH // 2 - 100, self.HEIGHT // 2 + 50, 80, 30],
            0,
            border_radius=5
        )
        replay_txt = self.font_small.render(" Replay", True, "white")
        self.surface.blit(
            replay_txt,
            (
                self.WIDTH // 2 - 100,
                self.HEIGHT // 2 + 50
            )
        )
```

```
232         # ------------------- DRAW QUIT BUTTON ---------------------------- #
233         quit_btn = pygame.draw.rect(
234             self.surface,
235             "black",
236             [self.WIDTH // 2, self.HEIGHT // 2 + 50, 80, 30],
237             0,
238             border_radius=5
239         )
240         replay_txt = self.font_small.render(" Quit", True, "white")
241         self.surface.blit(
242             replay_txt,
243             (
244                 self.WIDTH // 2 + 10,
245                 self.HEIGHT // 2 + 50
246             )
247         )
248 
249         # Copy the backbuffer into video memory
250         pygame.display.update()
251 
252         # Loop while waiting for user decision
253         while True:
254             for event in pygame.event.get():
255                 # print(event)
256                 if event.type == pygame.QUIT:
257                     pygame.quit()
258                     quit()
259         # ------------------- BUTTON CLICKED EVENTS ----------------------- #
260                 elif event.type == pygame.MOUSEBUTTONUP:
261                     if replay_btn.collidepoint(event.pos):
262                         # Stop any other music from playing
263                         pygame.mixer.stop()
264                         main()
265                     elif quit_btn.collidepoint(event.pos):
266                         pygame.quit()
267                         quit()
```

```python
# -------------------------- UPDATE BALL ----------------------------------#
    def update_ball(self):
        # Check for collision with left or right wall
        if self.ball_rect.left < 0 or self.ball_rect.right > self.WIDTH:
            # Reverse [0] x direction
            self.ball_speed[0] = -self.ball_speed[0]

        # Check for collision with top or bottom wall
        if self.ball_rect.top < 0 or self.ball_rect.bottom > self.HEIGHT:
            # Reverse [1] y direction
            self.ball_speed[1] = -self.ball_speed[1]

        # Move ball rect using speed list
        # x = speed[0], y = speed[1]
        self.ball_rect.left += self.ball_speed[0] * self.dt
        self.ball_rect.top += self.ball_speed[1] * self.dt

        # Ball hits bottom, player loses
        if self.ball_rect.bottom > self.HEIGHT:
            self.reset_game()
```

The Tractor Pong game, though it is very simple, contains many of the mechanics and physics of more complex games.

Time for you to see what you can create. Find some tutorials, start your game development career!

Assignment Submission

1. Attach the pseudocode.
2. Attach the program files.
3. Attach screenshots showing the successful operation of the program.
4. Submit in Blackboard.